LOVE WILL find answers.
Natalia went from having up to 30 seizures a day, to none. Discover...
SPONSORED BY CHILDREN'S MERCY HOSPITAL
See More

Test coverage analysis

07 MAY

I enjoy writing tests for my code for the obvious reasons: they build my confidence on the correct functionality of the software and also tend to drive the design in a more readable and well-structured direction. However, until recently I had been limited to performing only statement coverage analysis on my tests which is why I got very excited when I was able to start tracking both branch coverage and condition coverage in our recent projects.

Below is a short introduction to the different types of coverage analysis you can perform with currently available tools.

Set up

We'll start with a virtual environment to contain the example package and run the tests. We'll also install Nose (http://somethingaboutorange.com/mrl/projects/nose/1.0.0/), coverage (http://nedbatchelder.com/code/coverage/) and instrumental (http://pypi.python.org/pypi/instrumental) in the environment.

```
$ virtualenv-2.6 analysis
New python executable in analysis/bin/python2.6
Also creating executable in analysis/bin/python
Installing distribute.....................done.

$ cd analysis
$ ./bin/pip install nose coverage instrumental
```

Inside the virtualenv we have a Python package called "example" with two modules: *lib.py* and *tests.py*. The lib.py module contains the following function that we will test and the tests.py will contain the test cases.

```
def func(a, b):
    value = 0
    if a or b:
        value = value + 1
    return value
```

Although the function is very simple it will allow us to demonstrate the different coverage analysis tools.

Statement coverage

Statement coverage is probably the simplest of the three and its goal is to keep track of the source code lines that get executed during the test run. This will allow us to spot obvious holes in our test suite. We'll add the following test function in the tests.py module.

```
def test1():
    from example.lib import func
    assert func(True, False) == 1
```

With both nose (http://somethingaboutorange.com/mrl/projects/nose/1.0.0/) and coverage (http://nedbatchelder.com/code/coverage/) installed in the virtualenv we can run the tests with statement coverage analysis with

```
$ ./bin/nosetests -v --with-coverage example
example.tests.test1 ... ok

Name          Stmts   Miss  Cover   Missing
-------------------------------------------
example           0      0   100%
example.lib       5      0   100%
-------------------------------------------
TOTAL             5      0   100%
----------------------------------------------------------------------
Ran 1 test in 0.002s

OK
```

REPORT THIS AD

As we can see above this single test managed to achieve 100% statement coverage in our example package. Next, let's add branch analysis in the mix.

Branch coverage

The purpose of branch coverage analysis is to keep track of the logical branches in the executing of the code and to indicate whether some logical paths are not executed during the test run. Even with 100% statement coverage is rather easy to have less than 100% branch coverage.

With nose there is unfortunately no command-line switch we can use to activate branch coverage tracking so we will create a *.coveragerc* file in the current directory to enable it. The *.coveragerc* file contains the following

```
[run]
branch = True
```

In our function we have a logical branch (the if-statement) and currently our tests only exercise the True-path as can be seen when we run the tests with branch coverage enabled.

```
$ ./bin/nosetests -v --with-coverage example
example.tests.test1 ... ok

Name          Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------
example           0      0      0      0   100%
example.lib       5      0      2      1    86%
-------------------------------------------------------
TOTAL             5      0      2      1    86%
----------------------------------------------------------------------
Ran 1 test in 0.002s

OK
```

The output tells us that in *example.lib* we have one partial branch (BrPart) which reduces the coverage in that module to 86% in this case. We'll now add another test cases in *tests.py* which exercises the False-path of the if-statement.

```
def test1():
    from example.lib import func
    assert func(True, False) == 1

def test2():
    from example.lib import func
    assert func(False, False) == 0
```

Rerunning the tests with branch coverage tracking will show that we've now covered all logical branches.

```
$ ./bin/nosetests -v --with-coverage example
example.tests.test1 ... ok
example.tests.test2 ... ok

Name          Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------
example           0      0      0      0   100%
example.lib       5      0      2      0   100%
-------------------------------------------------------
TOTAL             5      0      2      0   100%
----------------------------------------------------------------------
Ran 2 tests in 0.002s

OK
```

ADVERTISEMENT

Discuss
Share

REPORT THIS AD

At this point things are looking much better. We have 100% statement and 100% branch coverage in our tests. There is still one part of our function which is not fully covered by our tests which is the compound boolean expression in the if-statement. For this we need condition coverage analysis.

Condition coverage

The purpose of condition coverage analysis is to track the execution paths taken while evaluating (compound) boolean expressions.

At the logical branch level our if-statement can take one of two logical paths which we already have tests for. However, this decision on the branch is only taken once the compound boolean expression has been evaluated. Within a boolean expression the computation may take up to 2^n possible paths (because of Python's short circuiting semantics the number of possible paths is actually less). These possible paths are probably easiest to think about using truth tables which show all the possible combinations. For our two part, "*a or b*", expression we can write the following truth table

a	**b**	**a or b**
False	False	False
False	True	True
True	False	True

a	b	a or b
True	True	True

Because in Python *and* and *or* are short-circuit operators (meaning their arguments are evaluated from left to right, and evaluation stops as soon as the outcome is determined) the (True, False) and (True, True) lines in our truth table are equivalent which reduces the truth table to three possible logical paths. Looking at the current test code we can see that even with 100% statement and 100% branch coverage we are missing an execution path in our function. We can verify this by using instrumental (http://pypi.python.org/pypi/instrumental) to run our tests which keeps track of conditions and shows the missing lines in our truth table.

```
$ ./bin/instrumental -rs -t example ./bin/nosetests example -v --with-covera
example.tests.test1 ... ok
example.tests.test2 ... ok

Name          Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------
example           0      0      0      0   100%
example.lib       5      0      2      0   100%
-------------------------------------------------------
TOTAL             5      0      2      0   100%
----------------------------------------------------------------------
Ran 2 tests in 0.002s

OK
example.lib: 4/5 hit (80%)

----------------------------
Instrumental Coverage Summary
----------------------------

example.lib:3 < (a or b) >

T * ==> True
F T ==> False
F F ==> True
```

REPORT THIS AD

We can see the output of instrumental at the bottom. For each boolean expression instrumental prints the location and the expression followed by the corresponding truth table. The truth table contains the possible values for the expression followed by "==> *True*" if the corresponding logical path was executed and "==> *False*" if not. In the above we can see that our current tests exercise the (True, *) and (False, False) combinations but the (False, True) case is missing. instrumental denotes the short-circuited case with an asterisk (T *) meaning that the second condition was not executed at all.

We now add a third test case to exercise the missing path.

```
def test1():
    from example.lib import func
    assert func(True, False) == 1

def test2():
    from example.lib import func
    assert func(False, False) == 0

def test3():
    from example.lib import func
    assert func(False, True) == 0
```

and rerun the tests

```
$ ./bin/instrumental -rs -t example ./bin/nosetests example -v --with-covera
example.tests.test1 ... ok
example.tests.test2 ... ok
example.tests.test3 ... ok

Name          Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------
example           0      0      0      0   100%
example.lib       5      0      2      0   100%
-------------------------------------------------------
TOTAL             5      0      2      0   100%
----------------------------------------------------------------------
Ran 3 tests in 0.002s

OK
example.lib: 5/5 hit (100%)

-----------------------------
Instrumental Coverage Summary
-----------------------------
```

Now we've finally managed full statement, branch and condition coverage on our function!

Conclusions

Having good tests and even 100% statement coverage is very good but it should only be considered the beginning and not the final goal in any project. With existing tools it is possible to analyze and improve test coverage with minimal effort.

Neither coverage (http://nedbatchelder.com/code/coverage/) nor instrumental (http://pypi.python.org/pypi/instrumental) is dependent on nose or any particular test runner so you should be able to use them in a variety of environments. For Zope/Plone development I can particularly recommend coverage over z3c.coverage (http://pypi.python.org/pypi/z3c.coverage). With coverage you can also generate statistics in XML format (for both statement and branch coverage) which can be monitored and tracked in systems such as Jenkins (http://jenkins-ci.org/).

For me condition coverage analysis was the most interesting technique of the three mostly because I was already familiar with the other two. Even before using coverage to automatically track branch coverage it was part of my test writing process to manually review the code in terms of the logical branches to make sure they were covered by tests. However, having an automated tool to do that is a big benefit. The instrumental (http://pypi.python.org/pypi/instrumental) package is still in development but in the cases I've used it it has done its job well and revealed interesting holes in our tests. If you're aware of other tools that provide condition coverage analysis I'd be interested in learning about them.

7 Comments

Posted by Kai Lautaportti on May 7, 2011 in python, software engineering

Tags: coverage analysis, python, testing

7 responses to "*Test coverage analysis*"

Brandon Craig Rhodes

May 9, 2011 at 4:39

Wow — I use "coverage" all the time, but had never even heard of "instrumental"! Thanks for pointing it out. Given the bare description on PyPI, I am not sure that I would ever have caught on to exactly what it was for.

Oh: and, "easy_install" is deprecated, and you will get better and cleaner results (as well as the ability to un-install later!) if you use "./bin/pip install " instead. It should come standard with all modern installs of virtualenv. Enjoy!

Reply

Kai Lautaportti

May 9, 2011 at 9:30

Hi Brandon

Thanks for the tip. I rarely use easy_install so I wasn't aware of the current best practices concerning easy_install/pip. I've modified the post to use pip instead.

Reply

Alex Marandon

May 12, 2011 at 10:38

Thanks a lot for this instructive article. I have a question: where does the "5 hits" come from in instrumental output? I would understand 4, or 3 if we don't count combinations made redundant by short circuiting semantics. But why 5?

Reply

Kai Lautaportti

May 12, 2011 at 11:33

Hi Alex

That's a good question. I checked the instrumental source code and the reason for five hits is that instrumental keeps track of both the results of the boolean expression and the conditions within the expression. In other words it performs both branch analysis and condition analysis.

So the total of five hits consists of two hits from the if-statement and three hits from the condition combinations.

Reply

Alex Marandon

May 12, 2011 at 11:37

All right that makes sense now. Thanks.

Reply

Matthew Desmarais

July 27, 2011 at 2:46

Thanks for including instrumental in your very cool write-up!

The thing it needs most of all at this point is use and feedback, and your post seems to help in both regards. I'd love to hear more about what users want/need from the tool, so please file bugs on bitbucket [0]! I'd love to improve the tool, but I'm still trying to figure out what to do next; bugs will help.

[0] https://bitbucket.org/desmaj/instrumental

Reply

Pulkit Garg

December 7, 2012 at 8:03

Worth Reading…

Reply

Create a free website or blog at WordPress.com.

Entries (RSS) and Comments (RSS)